

Mental-Model-Content

Before You Touch a Single Tool, Read This

Most people think they know what AI is.

They've used ChatGPT. They've typed a question, gotten an answer, maybe felt a little impressed, maybe a little underwhelmed. And they've filed it under "useful sometimes" and moved on with their life.

If that's you -- good. You're in the right place. Because what you're about to learn has almost nothing to do with what you've experienced so far.

I'm going to share three distinctions with you. They're not complicated. But they will fundamentally change how you think about AI, about the tools we're going to use, and about what's actually possible when you stop chatting with AI and start *building* with it.

These three ideas are the reason this program exists. Get them, and everything that follows will make sense. Miss them, and you'll spend the next few weeks wondering why we're not just using ChatGPT like everyone else.

Here's What Most People Get Wrong About AI

Let me give you a number that should stop you in your tracks.

88% of organizations say they've adopted AI. Only 6% are seeing meaningful profit impact.

That's a McKinsey survey -- thousands of companies, 2025. Eighty-eight percent are *using* it. Six percent are getting *results* from it.

Now, ask yourself this: What is happening in the gap between the 88% and the 6%?

Because the 88% are not stupid. They've bought the tools. They've run the pilots. They've sent their people to prompt engineering workshops. And they're still stuck.

The answer is that the 88% are doing the wrong thing with AI. They're chatting with it when they should be building with it. They're optimizing the wrong layer. And they're using tools designed for conversation when

they need tools designed for construction.

That's what the three distinctions below are about. Not which AI is "better." Not which tool has more features. But which *mode* you're operating in -- because mode determines everything.

Distinction #1: Coding CLIs vs. Chat Interfaces

Every major AI platform has two modes. Two completely different ways to use it. And almost nobody talks about the difference.

Mode one: the chat interface. ChatGPT, Claude.ai, Gemini -- you open a browser, type a question, get a response. This is what 99% of people think AI is. It's useful. It's impressive. And it's the wrong mode for what we're doing.

Mode two: the coding CLI. Claude Code, OpenAI's Codex CLI, Google's Gemini CLI, and others -- these tools run on your computer, inside your file system, with access to your actual projects and documents. They don't just answer questions. They read your files, edit your documents, create systems, and build things that persist after the session ends.

Every major AI company has built both. The chat interface is the one they market. The coding CLI is the one that changes how you work.

Here's the right question: **What happens the day AFTER you get the answer?**

With a chat interface, you type a question, get a response, close the tab, and tomorrow you start from scratch. Every conversation is a fresh start. There's no memory of what you built yesterday. No ability to save the instructions that worked. No way to hand off what you learned to a system that keeps running without you.

A coding CLI is different. It lives on your machine. It reads your files. It edits your documents. It remembers your project context across sessions. When you build something today, it exists tomorrow. And the day after that. And the day after that.

The analogy that makes this click: Imagine you need to get across town every morning. A chat interface is like asking a stranger for directions each day. You get a fine answer. Maybe even a great answer. But tomorrow morning, you're asking again. A coding CLI is like building a road. The first day takes longer. But from then on, you just drive.

One approach is *tool-mode*: bring a task to the AI, get an answer, leave. The other is *infrastructure-mode*: build persistent systems that compound over time.

We use Claude Code specifically -- not because it's the only coding CLI, but because it's the best one for what we're building. The distinction that matters, though, is the paradigm: CLI tools that operate on your system vs. chat interfaces where you type and copy-paste.

Here's where it gets interesting.

The entire AI industry has been selling you on something called "prompt engineering" -- the art of crafting clever instructions to get better answers. And prompt engineering isn't useless. But research from some of the smartest people in AI has broken down where results actually come from:

- **15%** comes from which AI model you're using
- **10%** comes from the prompt -- your clever instructions
- **75%** comes from everything else: the context you provide, the architecture around the AI, the system design, the data, the configuration

Read that again. 75% of your results come from the stuff nobody is teaching.

Most AI training teaches you to optimize the 10%. The prompt. The clever question. And they ignore the 75% that actually determines whether you get real results or just impressive-sounding responses.

This program teaches the 75%.

A coding CLI is the tool that lets you build in that 75% -- the persistent instructions, the project context, the reusable configurations that turn a one-time answer into a compounding system.

That's why we use one. Not because it's "better" than a chat interface. Because it operates in a completely different mode.

Distinction #2: Obsidian vs. Google Docs, Notion, Evernote

Here's a sentence I never thought I'd write: the most important decision in your AI setup is where you keep your files.

Not which AI model you use. Not which subscription you buy. Where your files live.

Let me tell you how I learned this the hard way.

I was a master power user of Evernote. Loved it. Used it for everything. And then I started working seriously with AI and hit a wall almost immediately. I couldn't get my notes out of Evernote and into AI easily. I couldn't get AI's output back into Evernote easily. Every interaction required me to manually drag files in, copy results out, paste them somewhere. The tool I'd relied on for years was suddenly a bottleneck.

So I moved to Notion, hoping it would be better. And it was -- a little. Notion had an integration that let AI pull files and work on them directly. But it didn't always work. Sometimes the connection failed. Sometimes files didn't sync correctly. And here's the part that really burned me: when it didn't work, I was dead in the water. There was no copy of my files on my computer. Everything lived in Notion's cloud. If the integration broke, I had nothing to work with.

I stumbled on Obsidian almost by accident. And if I'm being honest, I didn't even like it at first. I was so used to Evernote that Obsidian felt like a step backward. Simpler. Less polished. Not what I was used to.

But the more I used Obsidian with AI, the more I fell in love with it.

AI never had a problem getting to the files. Every single file was perfectly formatted for AI -- plain text, no proprietary format, no conversion needed. AI could read files, write files, create new files, and organize them with zero friction. It just worked. Every time.

An Obsidian vault is just a folder on your computer. That's it. No cloud. No proprietary format. Just a folder full of plain text files sitting on your hard drive, right next to your photos and your spreadsheets. You own those files. They're on YOUR machine. You can open them with any text editor on earth. And they'll still be readable in fifty years, regardless of what happens to any company.

And because those files are plain text on your computer, your coding CLI reads them directly. Your notes, your documents, your project files, your business context -- the AI sees all of it, instantly, without any export, conversion, or API. Your vault IS the context.

I think one of the major reasons I'm so far ahead of most people with AI is simply that I stumbled on Obsidian early. It gave my AI long-term permanent memory before anyone else had figured that out. To this day, most people have no clue about the kind of memory system you can build when you have Obsidian connected to your AI. If I had to choose one program -- one that went from completely unknown to me two years ago to the single program I must have on my computer in order to work -- it would be Obsidian.

The analogy that makes this click: There's a line I love from a researcher named Eva Keiffenheim. She said we've gotten really good at capturing and organizing information. We've built beautiful digital warehouses -- everything tagged, sorted, searchable. But we're treating these systems like warehouses when we should be treating them like *forges*.

A warehouse stores things. You walk in, find what you need, walk out. A forge *transforms* things. Raw material goes in. Something useful comes out.

Google Docs, Notion, Evernote -- they're warehouses. You store information there. It sits. You search for it when you need it. Maybe you share it with a colleague.

Obsidian -- connected to a coding CLI -- is a forge. Your knowledge goes in, and the AI transforms it. It pulls from your notes to write a proposal. It references your meeting transcripts to prepare a briefing. It connects patterns across hundreds of files that you would never have linked yourself.

The shift is not from one note-taking app to another. It's from storing knowledge to *using* knowledge.

And in a world where AI can process information instantly, the bottleneck is no longer "do I have enough information?" The bottleneck is "is my information in a form that AI can actually use?" Plain files on your computer solve that problem. Someone else's cloud doesn't.

Distinction #3: Getting an Answer vs. Building a System That Answers

This is the distinction that separates the 6% from the 88%.

And it starts with a question most people never ask: **Who is the assistant?**

Think about how most people use AI right now. They open a chat interface. They need AI to work on a document, so they go find the document, drag it into the chat, wait for the response, copy the output, go put it where it belongs, come back, drag in the next file, repeat.

The AI sits in one place. The human runs all over.

The AI does the thinking. The human does the fetching, the carrying, the filing, the organizing. The human is the assistant to the AI.

Now think about how it works with a coding CLI. You sit at your computer. You tell the AI what you need. The AI goes and grabs the files. It reads them. It works on them. It creates new files, organizes them, puts them where they belong, and tells you when it's done. You bark the orders. The AI does the running.

You take back your power.

That's not a small difference. That's a complete inversion of the relationship. In one mode, you serve the AI. In the other, the AI serves you. And when the AI is the one doing the running, you can do something that's impossible in chat mode: you can build systems that keep working after the conversation ends.

Picture two people. Both are using AI. Both are smart. Both work hard.

Person A asks AI for answers. They open a chat, type a question, get a response, copy it somewhere, close the tab. Tomorrow they do it again. Different question, same process. Each interaction is independent. Nothing carries forward. Every day starts from zero.

Person B builds a system that answers. They write persistent instructions that tell the AI how they work, what they care about, what their business looks like. They organize their files so the AI always has the right context. Each session builds on the last. The system gets smarter every time they use it.

Person A's output is *linear*. Double the time, double the output. Stop working, output stops.

Person B's output *compounds*. The hundredth session is ten times faster than the first, because the system already knows everything. It works even when Person B isn't there.

Here's a real example. A guy named Alex McFarland spent about an hour building context profiles for his AI system -- descriptions of his voice, his business, his ideal clients. Not prompts. Context. Structured information about who he is and what he does.

Then he rebranded his entire business in two days. Fifteen-plus pieces of professional content. Not because he wrote better prompts. Because he built a system that already knew him. Everything it produced was already aligned.

One hour of building replaced weeks of asking for answers.

And here's the uncomfortable truth: most people never build the system. They just keep asking for answers. They open the AI, type a question, get a response, and feel productive. But nothing accumulates. Nothing compounds. Tomorrow is the same as today. And they're still the one running around fetching files.

The 88% are asking for answers. The 6% have built systems that answer.

This program teaches you to build the system.

What This Means for You

I want to be straight with you about something.

I built this system myself. I use it every day. The AI tools you're about to set up are the same ones running in my business right now -- reading my files, executing my workflows, building content, managing projects. This is not theory I read in a book and decided to teach. It's infrastructure I built, tested, broke, rebuilt, and now rely on.

I'm not going to pretend the setup is effortless. There's a learning curve. You'll hit moments where you're frustrated, where it's not working the way you expected, where you want to go back to the tools you already know.

Here's what I've learned about those moments: you cannot learn unless you push through them. The frustration IS the learning. And on the other side of that frustration is a system that makes you feel -- and I'm not exaggerating here, because I've heard the same thing from Todd Brown, Mike Filsaime, Russell Brunson, and half a dozen other people who've built similar setups -- *invincible*.

Not because the AI does everything for you. But because the right system, set up the right way, with the right context, multiplies everything you already know. Your expertise doesn't become less valuable. It becomes the single most important input to a system that amplifies it beyond anything you could do alone.

Research backs this up. A study of 667 people found that working effectively with AI is a *separate skill* from being good at your job. Years of experience, advanced degrees, deep expertise -- none of it predicted who would get the best results from AI. Some average performers saw massive improvements. Some top performers barely improved at all.

The difference was not intelligence. It was not domain expertise. It was the skill of building systems that let AI work with what you know.

That's the skill this program teaches.

What Happens Next

Part 2 is where you set up Claude Code on your computer. It's a step-by-step walkthrough -- no guesswork, no assumptions about what you already know.

You don't need to be technical. You don't need to have used a terminal before. You don't need any programming experience. If you can follow instructions on a screen, you can do this.

Part 2 is the setup. But Part 1 -- what you just read -- is *why* the setup matters. You're not installing software for the sake of installing software. You're laying the foundation of a system that compounds.

The road, not the directions. The forge, not the warehouse. The system, not the answer.

You're Done When...

You've finished Part 1 when you can answer these three questions in your own words:

1. **Why a coding CLI and not a chat interface?** (Hint: it's not about which AI is smarter.)
2. **Why Obsidian and not Google Docs?** (Hint: it's about what AI can actually read.)
3. **What's the difference between getting an answer and building a system that answers?** (Hint: one compounds, the other doesn't. And only one puts you in charge.)

If you can answer those three -- even roughly, even in your own shorthand -- you have the mental model. You're ready for Part 2.

Let's build.